

Goal: This pipeline aims to answer the following biological question: Whether these correlated enhancers of a given gene are driven by TEs (i.e., overlapped with TEs). In addition, this pipeline also tests if highly correlated enhancers related to a given gene would form a network or networks.

Example data: The testing dataset is a small mouse brain single-cell multiome dataset (221.2 MB) and can be downloaded from 10X Genomics website (<https://www.10xgenomics.com/datasets/fresh-embryonic-e-18-mouse-brain-5-k-1-standard-1-0-0>). Key metrics of the dataset can be found on the website.

We will use the processed data for testing the pipeline. Information about cell barcodes, features (gene names and locations, as well as peak locations) can be found in the folder named "filtered_feature_bc_matrix".

Pipeline walkthrough: All, except step 1, are implemented in python >= 3.7

Step 1 (required R version 3.4 or higher): Using the Cicero library to quantify co-accessibility scores of peaks near the gene of interest.

```
library(monocle3)
library(cicero)
library(Matrix)

indata <- readMM(paste(data_path,"filtered_feature_bc_matrix/matrix.mtx.gz", sep="",
collapse=NULL))
# binarize the matrix
indata@x[indata@x > 0] <- 1

# format cell info
cellinfo <- read.table(paste(data_path, "filtered_feature_bc_matrix/barcodes.tsv.gz",
sep="", collapse=NULL))
row.names(cellinfo) <- cellinfo$V1
names(cellinfo) <- "cells"

# get features
features=read.table(paste(data_path, "filtered_feature_bc_matrix/features.tsv.gz",
sep="", collapse=NULL), header=F,
row.names=1, sep="\t")
peaks_only <- rownames(features)[features$V3 == "Peaks"]
gene_only <- rownames(features)[features$V3== "Gene Expression"]

# create a function to get highly correlated enhancers of target gene (i.e., corr
coefficient >=0.15)
res=gene_enhancer_corr(features)
write.csv(res, "results/enhancers_corr_gt0.15_with_Macrod2.csv", row.names = FALSE)

#Format peak info
peakinfo <- read.table(paste(data_path,"filtered_feature_bc_matrix/peaks.bed", sep="",
collapse=NULL))
names(peakinfo) <- c("chr", "bp1", "bp2")
peakinfo$site_name <- paste(peakinfo$chr, peakinfo$bp1, peakinfo$bp2, sep="_")
peakinfo$site_name <- sub("_", ".", peakinfo$site_name, fixed=T)
peakinfo$site_name <- sub("-", "-", peakinfo$site_name, fixed=T)
row.names(peakinfo) <- peakinfo$site_name

row.names(indata) <- row.names(features)
colnames(indata) <- row.names(cellinfo)

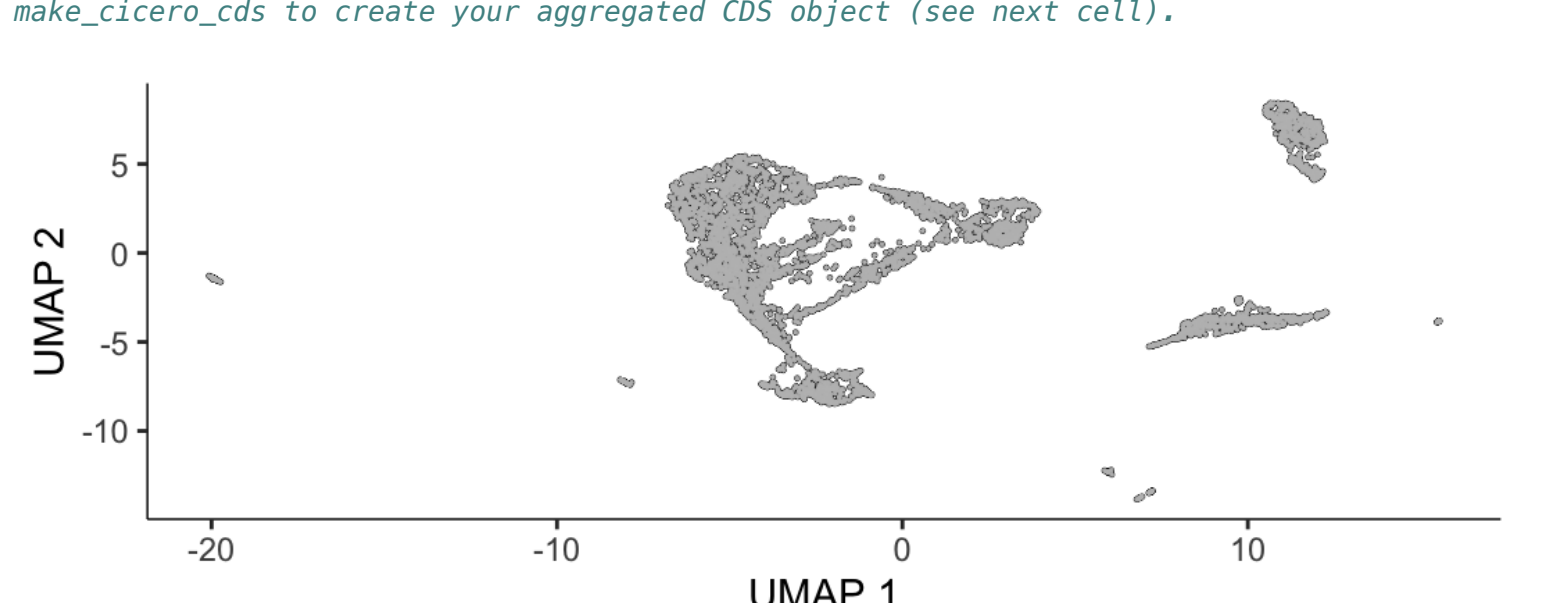
# only use rows that are peaks
new_indata <- indata[peaks_only, ]
input_cds = suppressWarnings(new_cell_data_set(new_indata, cell_metadata = cellinfo,
gene_metadata = peakinfo))

# ensure no peaks included with zero reads
input_cds <- input_cds[Matrix::rowSums(exprs(input_cds)) !=0,]

set.seed(2017)
input_cds <- monocle3::detect_genes(input_cds)
input_cds <- estimate_size_factors(input_cds)
input_cds <- preprocess_cds(input_cds, method="LSI")
input_cds <- reduce_dimension(input_cds, reduction_method = 'UMAP', preprocess_method =
"LSI")

# plot umap
plot_cells(input_cds)

# Once you have a reduced dimension coordinate map, you can use the function
make_cicero_cds to create your aggregated CDS object (see next cell).
```



Next, we prepare a cicero data object and plot the connections between peaks for a selected gene

```
umap_coords <- reducedDims(input_cds)$UMAP
# make a cicero data object using the umap coordinates
cicero_cds <- make_cicero_cds(input_cds, reduced_coordinates = umap_coords)

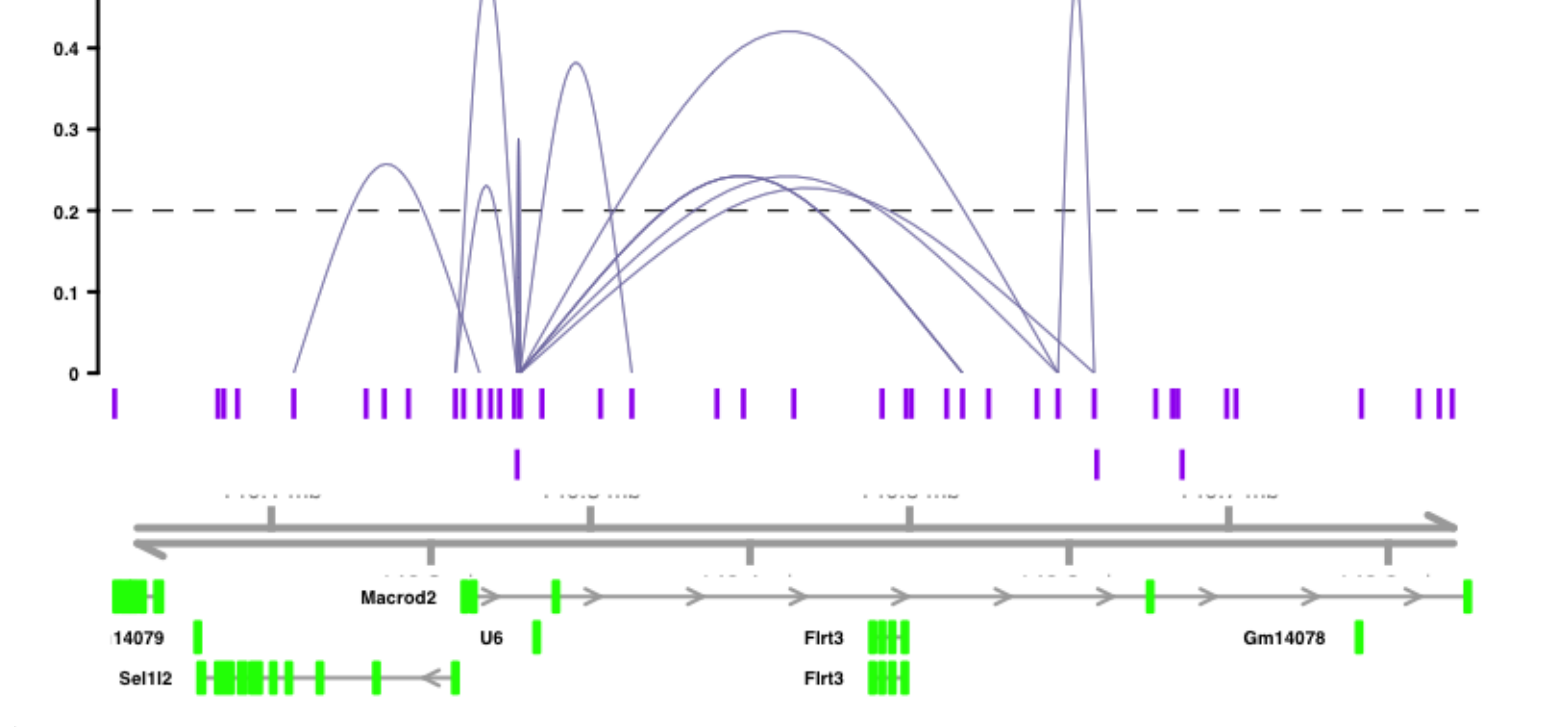
# Here we use a mouse ATAC-seq data from 10X multiome as input, so we go with the mouse
genom.
# Change to the respective genome if data is of a different species
data("mouse.mm9.genome")

# use only a small part of the genome and a small sample size to test the connections
between peaks
# Specify the chromosome where the target gene (e.g., chromosome 3 for Macrod) locates
sample_genome <- subset(mouse.mm9.genome, V1 == "chr2")

# run on 2 samples
conns <- run_cicero(cicero_cds, sample_genome, sample_num = 2)
conns_sub <- conns[conns$coaccess >=0.2, ]
# drop nan
comp_cases=conns_sub[complete.cases(conns_sub), ]

# Visualizing Cicero Connections for chr 2
# Download the GTF associated with this data (mm9) from ensembl and load it
# using rtracklayer
# download and unzip
temp <- tempfile()
download.file("ftp://ftp.ensembl.org/pub/release-
65/gtf/mus_musculus/Mus_musculus.NCBI37.65.gtf.gz", temp)
gene_anno <- rtracklayer::readGFF(temp)
unlink(temp)

# rename some columns to match requirements
gene_anno$chromosome <- paste0("chr", gene_anno$seqid)
gene_anno$gene <- gene_anno$gene_id
gene_anno$transcript <- gene_anno$transcript_id
gene_anno$symbol <- gene_anno$gene_name
plot_connections(conns, "chr2", 140000000, 140856988,
gene_model = gene_anno,
coaccess_cutoff = .2,
connection_width = .5, peak_color="purple", gene_model_color="green",
collapseTranscripts = "longest" )
```



Finally, we run cicero and save those enhancer-pair with co-accessibility scores >= 0.2

```
# Use a large samples and the whole genome for real
conns <- run_cicero(cicero_cds, mouse.mm9.genome, sample_num = 100)
# Here we select connections with co-accessibility scores >= 0.2
conns_sub <- conns[conns$coaccess >=0.2, ]
# save the subset data for the sunsequent step
write.csv(conns, "results/whole_genome_coaccess_gt0.2.csv", row.names=FALSE)
```

The following steps are done in python, so import all the needed packages first

```
In [1]: import pandas as pd
import numpy as np
import random
from pybedtools import BedTool
import scipy.sparse
from fuc import pybed
from scipy.io import mmwrite
from scipy.sparse import csr_matrix
from scipy.sparse import coo_matrix
from collections import defaultdict
import os
import glob
import csv
import gzip
import matplotlib.pyplot as plt
import network_visualization as nv
import steamer_enhancer as se
```

Step 2: Construct networks for highly correlated enhancers related to a pre-selected target gene

First, we select enhancers highly correlated with the expression of Macrod2 and from these enhancers, we further select those with high co-accessibility (i.e., co-accessibility score >=0.2)

```
In [104... coess=pd.read_csv("results/marods2_enhancer_coaccess_gt0.2.csv")
# make a dataframe of enhancers and gene expression
dfam=coess[coess["Peak1"].isin(sigE_peaks)].loc[:, ("Peak1", "Peak2", "coaccess")]
sub.to_csv("results/sig_enhancers_corr_Macrod2_their_sig_coaccessibility.csv")

In [2]: # Gene of interest: MACROD2, a protein-coding gene associated with disease, such as hypogonadotropic
# MACROD2 is located at chromosome 2, 140395308-140395595
# select enhancers locate within 500kn up- and down-stream of MACROD2 and with co-accessibility score >= 0.2
output_dir = 'results/'
os.makedirs(output_dir,exist_ok=True)
#whole_gem=pd.read_csv("results/whole_genome_gt0.2.csv")
#df=pd.read_csv("results/whole_genome_gt0.2.csv").dropna()
#df=pd.read_csv("results/sig_enhancers_corr_Macrod2_their_sig_coaccessibility.csv").dropna()
# extract correlated enhancers of the target gene, Macrod 2 (on chromosome)
#macrod2_df=df[df["Peak1"].str.contains("chr2")]
#macrod2_df.to_csv("results/enhancers_corr_gt0.15_with_Macrod2.csv")

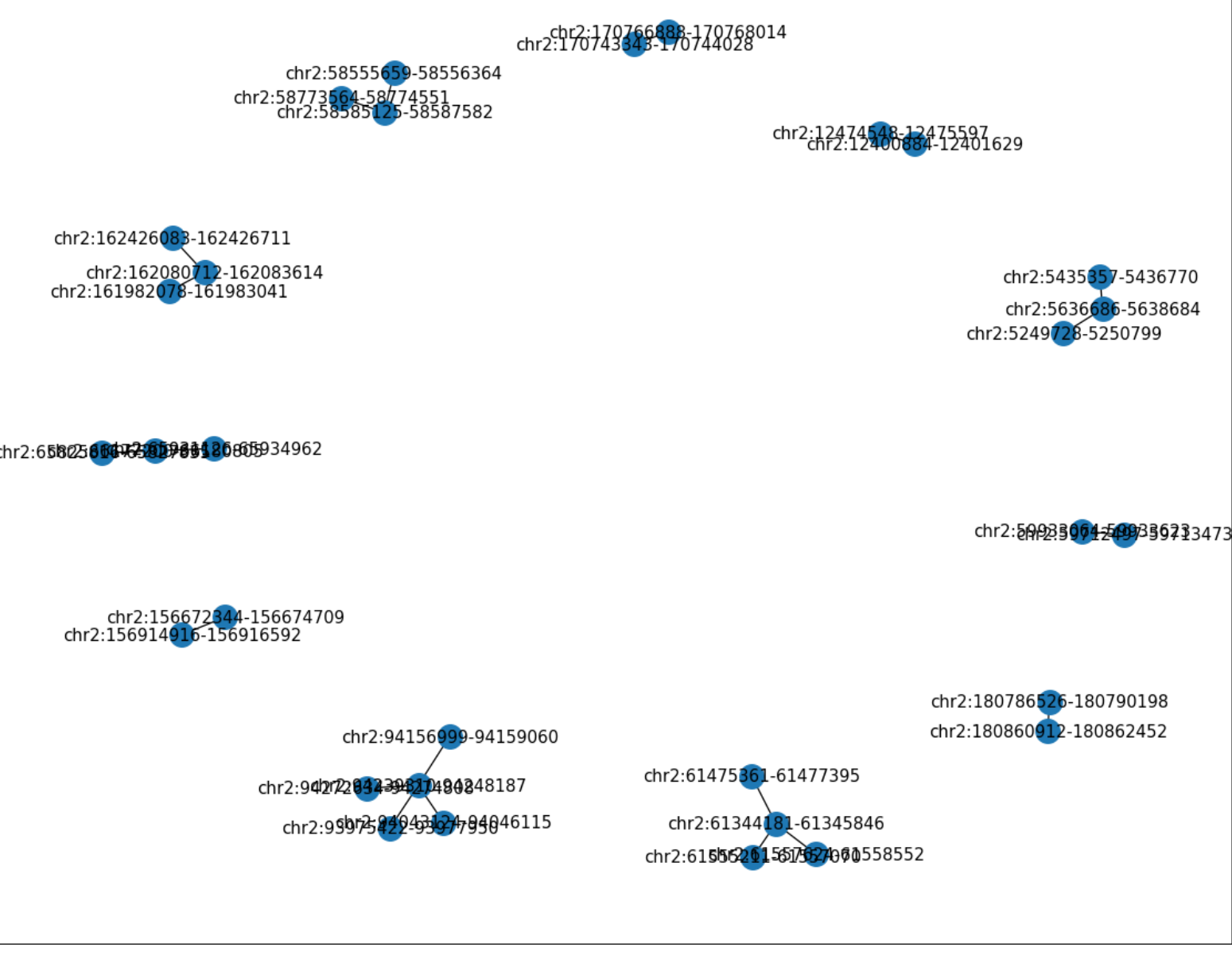
# concatenate vertically these enhancers to the dataframe
new_df = se.get_enhancers("results/sig_enhancers_corr_Macrod2_their_sig_coaccessibility.csv")

# prepare significant enhancers in the bed format
nearby_enhancers, barcode= se.get_nearby_enhancers(new_df, "chr2", 140395308, 140395595)
```

Step 3:Visualize networks

```
In [3]: # Since there are still too many enhancer-pairs that makes network visualization difficult, we will
# The definition of strongly correlated enhancer pairs will be provided by users as min_high_corr,
```

```
#filename="results/marods2_enhancer_coaccess_gt0.2.csv"
#filename="results/sig_enhancers_corr_Macrod2_their_sig_coaccessibility.csv"
nv.network_plot(filename, 0.55)
```



Step 4: Prepare TE annotation file (in .bed) and intersect TEs and nearby_enhancers bed files

```
In [ ]: TEs=se.create_bed_for_TEs("mm10.nrph.hits.gz")
# read in TEs.bed and enhancer bed file, Frag.bed, which was generated in the previous step
TEs=BedTool("results/TEs.bed")
frags=BedTool("results/Frag.bed")
intersected_bed=se.intersection(TEs, frags)
```

Step 5: Compute TE-by-enhancer matrices

```
In [9]: barcode=pd.read_csv("results/Frag.bed", sep="\t", header=None).iloc[:, 3]
dfam=pd.read_table("mm10.nrph.hits.gz", sep="\t")
TE_fams=dfam["family_name"].unique().tolist()

unique_TEs_sparse_matrix,TE_fams_sparse_matrix,unique_TEs_list,TEs_fams,barcode_dict=se.make_cell_x
print(f"Number of enhancers present in the TE-by-enhancer matrix: {unique_TEs_sparse_matrix.shape}")
# save as sparse matrices
unique_TEs_sparse_matrix = se.convert_df_to_sparse(unique_TEs_sparse_matrix)
TE_fams_sparse_matrix = se.convert_df_to_sparse(TE_fams_sparse_matrix)

# save as 10X-like data format
output_dir = 'results/TE_Fam_matrix/'
file_path = 'barcodes.tsv'

output_dir = 'results/TE_Fam_matrix/'
os.makedirs(output_dir,exist_ok=True)
#start saving barcode info
file_path = 'barcodes.tsv'
#se.compress_tsx_file(file_path,output_dir,barcode_dict)
#starting saving the features
df = se.make_features_df(TEs_fams)
path = 'results/TE_Fam_matrix/features.tsv.gz'
se.save_df_as_gz(df, path)
#start saving the matrix
file_path = 'results/TE_Fam_matrix/matrix.mtx'
se.compress_sparse_matrix(TE_fams_sparse_matrix,file_path)
print('Files saved to TE_Fam_matrix directory')
#start on the unique TE info
output_dir = 'results/TE_Unique_matrix/'
os.makedirs(output_dir,exist_ok=True)
#start saving barcode info
file_path = 'barcodes.tsv'
se.compress_tsx_file(file_path,output_dir,barcode_dict)
#start saving the features
df = se.make_features_df(unique_TEs_list)
path = 'results/TE_Unique_matrix/features.tsv.gz'
se.save_df_as_gz(df,path)
#start saving the matrix
file_path = 'results/TE_Unique_matrix/matrix.mtx'
se.compress_sparse_matrix(unique_TEs_sparse_matrix,file_path)
print('Files saved to TE_Unique_matrix directory')
```

Number of enhancers present in the TE-by-enhancer matrix: (220, 3)

Files saved to TE_Fam_matrix directory

Files saved to TE_Unique_matrix directory

Summary (answer the biological questions and draw a conclusion if possible)

This tutorial takes a single-cell multiome mouse brain data as input and yields the following findings:

1. As expected, some significant enhancers are overlapped with TE sequences.
2. Although most enhancer-enhancer connections are unique, there are a few of enhancers that connect with more than one enhancers and thus form a small network.

Conclusion:

TEs are known for their abundance and repetition in the genome. Findings from the mouse brain data not only are in line with this view, but further support that significant enhancers of a given gene (e.g., MACROD2 in this tutorial) can overlap with TEs. In addition, some of the significant enhancers connect with multiple enhancers and thus form small networks. The enhancer that has more than one connection in each network is often considered as the hub, hinting its more influential role in gene expression.